

Critix – Documentação Técnica Completa

Plataforma moderna de avaliação de filmes e séries com foco em escalabilidade, engajamento social, gamificação e streaming baseado em tickets.

1. Visão Geral do Sistema

Critix é uma plataforma híbrida (web + mobile nativo) com recursos sociais, streaming com tickets, gamificação profunda e integração com TMDB para dados de mídia. O objetivo é escalar para milhares de usuários mantendo performance e segurança.

2. Arquitetura Geral

A arquitetura é modular, separando camadas por contexto de domínio:

- Domínio de Usuários e Perfis
- Domínio Social (curtidas, seguidores, avaliações, comentários)
- Gamificação (XP, níveis, missões, conquistas)
- Tickets e Streaming
- Catálogo (cache local incremental com dados do TMDB)
- Watchlist pública/privada
- Chat em tempo real
- Moderação automática via ML
- Internacionalização (pt-BR/en)

Futuro: migração para microserviços conforme crescimento.

3. Arquitetura Backend – Spring

Tecnologias:

- Java última versão LTS
- Spring Web, Security, Data, Validation
- Spring Cloud (futuro microserviços)
- OpenFeign para integrações externas
- PostgreSQL + Redis
- WebSockets para streaming + chat

Padrões estruturais:

- Controladores → Services → Domain → Repository
- Domínios desacoplados por módulos internos
- Estratégia de API Versioning (v1, v2...)
- Adapters para integrações (TMDB, AbacatePay)

4. Fluxo de Streaming + Tickets

O streaming é liberado através de tickets obtidos por missões:

Fluxo:

1. Usuário tenta assistir algo.
2. Sistema verifica ticket disponível.
3. Se sim → libera o player.
4. Usuário pode criar sala e convidar outros.
5. Chat em tempo real via WebSocket.
6. Consumo de ticket é registrado.

Tickets podem ser obtidos por:

- Check-in diário
- Conquistas
- Engajamento social

5. Modelo de Gamificação

Sistema baseado em progressão:

- XP por atividades sociais
- Níveis (progressão linear ou exponencial)
- Conquistas desbloqueáveis
- Missões diárias/semanais
- Recompensas em tickets

Camada dedicada: AchievementEngine

- Processamento assíncrono via fila

6. Integração com TMDb

Estratégia de cache incremental:

1. Consulta ao banco local (media_cache).
2. Se não existir → buscar TMDb.
3. Salvar resposta normalizada em media_cache.
4. Atualizar dados periodicamente via job.

Vantagens:

- Reduz latência
- Reduz dependência externa
- Escalável com shardings futuros

7. Estrutura de Banco de Dados (Resumo)

Tabelas principais (PostgreSQL):

users

- id, name, email, auth_provider
- language, country
- xp, level
- metrics (total_reviews, total_movies, total_series)

reviews

- id, user_id, media_id, rating, text, created_at

media_cache

- id, tmdb_id, title, metadata_json

watchlist

- id, user_id, media_id, visibility

achievements

- id, code, description, reward_type, reward_value

user_achievements

- user_id, achievement_id, unlocked_at

tickets

- id, user_id, amount

streaming_rooms

- id, owner_id, media_id, is_active

room_users

- room_id, user_id

chat_messages

- id, room_id, user_id, content, timestamp

8. Frontend Web e Mobile

Tecnologias:

- Web: React + Tailwind + Zustand/Redux
- Mobile: React Native ou nativo
- WebSockets para streaming/chat
- Internacionalização com i18next

Camadas:

- Components
- Features
- Store
- API Modules
- Hooks

9. Infraestrutura e DevOps

- Deploy containerizado com Docker
- Orquestração: Kubernetes (futuro)
- CI/CD: GitHub Actions
- CDN para assets
- Logs centralizados
- Observabilidade (Prometheus + Grafana)
- Rate limiting (gateway)
- Cache Redis

10. Segurança

- Login social via Better-Auth
- Rate-limiting para endpoints sensíveis
- Proteção anti-brute-force
- Sanitização de input
- Uso de JWT + refresh tokens

11. Moderação via ML

- Classificação automática de comentários
- Pipeline de moderação pré-postagem
- Sistema de flag manual complementar

Modelo inicial: classificação de toxicidade

12. Decisão: Monolito Modular ou Microserviços?

Recomendação:

- Iniciar com monolito modular (mais simples)
- Dividir por módulos internos de domínio
- Migrar para microserviços apenas se:
 - >50k usuários ativos
 - Latência crítica
 - Muito tráfego 24/7
 - Streaming crescer exponencialmente

Microserviços candidatos:

- Gamificação Engine
- Chat Server
- Media Cache Service
- Users/Auth

13. Diagramas – Descrições

Diagramas que serão incluídos em versão estendida:

- Diagrama de Casos de Uso (US)
- Diagrama de Classes por Domínio
- Modelo Banco (ERD)
- Fluxo TMDB Cache
- Fluxo de Streaming

14. Conclusão

Base sólida para refatoração completa do Critix,
com arquitetura escalável, modular e preparada para
gamificação e streaming em grande escala.